

MASTER BUILD PROMPT – Blockchain + ML Pharmaceutical Supply Chain Intelligence Platform

Paste this entire document as the opening brief to a coding AI (Claude Code, Cursor, etc.) to start building. It supersedes the tech stack in any previously uploaded synopsis/blueprint – the **product scope, roles, ML modules, and research design are unchanged**, only the implementation stack is modernized for a single-team, buildable-in-one-semester, portfolio-grade result.

0. WHAT YOU ARE BUILDING

A pharmaceutical supply-chain platform that:

1. Tracks a medicine batch through six roles – **Admin, Supplier, Manufacturer, Wholesaler, Distributor, Customer** – from raw material to end customer.
2. Anchors every critical event (batch creation, custody transfer, order confirmation, delivery) on the **Algorand blockchain** as a tamper-evident, verifiable record.
3. Runs three ML models on top of that data to make the system **proactive** instead of just a tamper-evident logbook:
 - **Module 1 – Fraud & Anomaly Detection** (Isolation Forest + XGBoost)
 - **Module 2 – Counterfeit Risk Scoring** (Random Forest + SHAP, metadata-only, no physical sample)
 - **Module 3 – Inventory Demand Forecasting** (LSTM vs. XGBoost Regression)
4. Runs a controlled **A/B research experiment**: does adding blockchain-derived behavioural features (transfer count, ownership hops, route deviation, timestamp gaps, verification failures) improve fraud-detection accuracy over conventional features alone? Report Precision/Recall/F1/ROC-AUC and test significance with McNemar's test ($\alpha = 0.05$).
5. Gives customers QR-code batch verification and gives admins a real-time intelligence dashboard (anomaly alerts, risk heatmap, demand forecasts, SHAP explanations).

Research baseline to position against: Abbas et al. (2020) used ML only for drug **recommendation**; this system is the first to use blockchain-behavioural features for active fraud detection, and the first to score counterfeit risk from metadata alone (no image/spectroscopy).

Hard constraints – do not violate:

- Never claim ML "proves" a medicine is counterfeit – only that it estimates risk.
- Never claim blockchain guarantees truthful physical-world input – only tamper-evident recording of whatever was submitted.
- Clearly separate real/simulated transaction data from synthetically injected anomaly/counterfeit-label data everywhere (code comments, dataset filenames, dashboard labels).

- Enforce strict role-based access control on every endpoint, not just in the UI.

1. MODERNIZED TECH STACK (and why)

The original spec split the system across ASP.NET/C#, MSSQL, and a separate Python Flask microservice glued together over plain REST – three ecosystems, three build toolchains, no shared types, and MSSQL licensing overhead for a project with zero MSSQL-specific need. Replace it with a ****single-language, type-safe, container-native stack****:

Layer	Choice	Why
Frontend	**Next.js 15 (React 19, TypeScript, App Router)** + **Tailwind CSS** + **shadcn/ui**	One framework for all six role portals + admin dashboard; server components for fast data-heavy pages; shadcn gives you accessible, themeable components instead of hand-rolled CSS Charts / Dashboard **Recharts** + **Mapbox GL** (risk heatmap) Lightweight, React-native, good with streaming data
Backend (business logic + blockchain + ML orchestration)	**FastAPI (Python 3.12)**	One backend language for everything – business logic, blockchain calls (<code>`py-algorand-sdk`</code> is Python-native), and ML inference all live in the same process/repo. Eliminates the ASP.NET↔Flask REST hop entirely. Async-native, auto-generates OpenAPI, pairs naturally with Pydantic for shared request/response schemas
ORM / DB access	**SQLAlchemy 2.0 (async) + Alembic migrations**	Type-safe models, versioned schema
Database	**PostgreSQL 16**	Free, battle-tested, native JSONB (great for storing raw SHAP output / feature vectors), full-text search built in, easy Docker/managed hosting (Supabase/Neon/RDS) vs. MSSQL's licensing and hosting friction
Cache / Queue	**Redis**	(cache + Celery broker) Cache dashboard aggregates; queue long-running ML training jobs and daily forecast refreshes
Background jobs	**Celery**	(or APScheduler if you want to stay lighter) Nightly demand-forecast regeneration, periodic anomaly re-scoring
Blockchain	**Algorand TestNet** , smart contracts in **Algorand Python (Beaker/PyTeal successor) via AlgoKit** , <code>`py-algorand-sdk`</code>	Kept as specified – Algorand's low fees, fast finality, and Python-native tooling are genuinely a good fit here; AlgoKit is the current recommended dev workflow (not raw PyTeal)
Auth	**JWT (access + refresh) via FastAPI** + **bcrypt/argon2**	password hashing, role claims in token Stateless, works cleanly across all six roles
ML libraries	**scikit-learn** (Isolation Forest, Random Forest, XGBoost via <code>`xgboost`</code> package), **PyTorch** (LSTM – more current and flexible than TF/Keras for a portfolio project) or <code>`statsforecast`/`xgboost`</code> as the tabular baseline, **SHAP** for explainability	Same models as spec'd, modern library choices
Data processing	**pandas, NumPy, polars (optional, faster for larger synthetic datasets)**	

| Experiment tracking | **MLflow** (lightweight local server) | Logs Experiment A vs B runs, metrics, and model artifacts – makes the A/B research claim auditable and reproducible, which matters a lot for a paper/thesis defense |

| ML serving | Served **inside the same FastAPI app** as versioned endpoints (`/ml/fraud/score`, `/ml/counterfeit/score`, `/ml/demand/forecast`), models loaded via `joblib`/`torch.save` at startup | Removes the separate microservice + REST hop from the original design while keeping ML logically modular in its own package (`app/ml/`) |

| QR codes | `qrcode` (Python) to generate, phone camera + `zxing`/`jsQR` on frontend to scan | |

| Realtime | **WebSockets** (FastAPI native) or Server-Sent Events for live anomaly alerts on the admin dashboard | |

| Containerization | **Docker + Docker Compose** (postgres, redis, backend, frontend, mlflow, algorand sandbox via AlgoKit LocalNet) | One `docker compose up` boots the entire stack |

| CI | **GitHub Actions** – lint (ruff, eslint), type-check (mypy, tsc), pytest, playwright e2e smoke test | |

| Hosting (for demo) | Frontend → **Vercel**; Backend + Postgres + Redis → **Railway/Render**; Algorand → **TestNet** (public, free) | Free/near-free tier friendly for a student project |

Net effect of the change: one language (Python) for backend+blockchain+ML instead of three stacks, one Postgres instance instead of MSSQL, ML models served in-process instead of a second hop, and a modern typed frontend instead of static HTML/CSS/JS. Functionally identical to the spec – same six roles, same three ML modules, same A/B experiment – just fewer moving parts and easier to actually ship solo or in a small team.

If your program specifically requires ASP.NET/C# and MSSQL for grading reasons, say so and I'll give you the equivalent prompt with that stack instead – but absent that constraint, use the above.

2. MONOREPO STRUCTURE

...

```

pharma-chain/
├── apps/
│   └── web/                                # Next.js frontend (all 6 role portals +
│       └── admin dashboard)
│           ├── app/
│           │   ├── (auth)/login, register
│           │   └── admin/                # dashboard, alerts, heatmap, forecasts,
│           └── user mgmt
│               ├── supplier/
│               ├── manufacturer/
│               ├── wholesaler/
│               ├── distributor/
│               └── customer/              # order medicines, scan QR, view
└── provenance

```

```

├── verify/[batchId] # public QR landing page (no login
required)
├── components/
├── lib/api-client.ts # typed fetch wrapper generated from
OpenAPI
├── ...
├── api/ # FastAPI backend
├── app/
├── core/ # config, security, deps
├── models/ # SQLAlchemy models (Participant, Batch,
Transaction, Order, MLScore...)
├── schemas/ # Pydantic request/response schemas
├── routers/ # auth.py, participants.py, batches.py,
orders.py, transactions.py, admin.py, verify.py
├── services/
├── blockchain/ # algorand_client.py, contracts/
(AlgoKit project), tx_writer.py
├── qr.py
├── ml/
├── fraud/ # feature_engineering.py,
train_isolation_forest.py, train_xgboost.py, experiment_ab.py
├── counterfeit/ # train_random_forest.py,
shap_explainer.py
├── demand/ # train_lstm.py, train_xgb_regressor.py
├── synthetic/ # anomaly_injection.py,
counterfeit_label_simulator.py
├── registry.py # loads trained artifacts, exposes
.predict()
├── workers/ # Celery tasks: rescoring, nightly
forecast refresh
├── main.py
├── alembic/
├── tests/
├── contracts/ # AlgoKit Algorand Python smart contract
project
├── smart_contracts/
├── participant_registry/
├── batch_registry/
├── custody_transfer/
├── tests/
├── ml_notebooks/ # exploratory Jupyter notebooks (EDA,
model iteration)
├── docker-compose.yml
├── .github/workflows/ci.yml
├── README.md
...

```

3. DATABASE SCHEMA (PostgreSQL, via SQLAlchemy models)

...

```

participants      id, role (enum:
admin|supplier|manufacturer|wholesaler|distributor|customer),

```


storing the resulting `algorand\_tx\_id`; treat this as a single logical operation with a retry/reconciliation job for the blockchain leg so a chain hiccup never silently drops a DB row (or vice versa).

4. BLOCKCHAIN LAYER — ALGORAND

- Use **AlgoKit** to scaffold the smart-contract project (Algorand Python, the current successor to raw PyTeal).
- Three logical contracts/apps:
 1. **Participant Registry** — admin-only registration of wallet addresses to roles.
 2. **Batch Registry** — create batch (mint an ASA representing the batch or store a hash of batch metadata), immutable production record.
 3. **Custody Transfer** — records each ownership hop with sender, receiver, timestamp; this is the on-chain source for `transfer\_count`, `ownership\_hops`, `timestamp\_gap\_hrs`.
- Run against **Algorand TestNet** for development/demo (free, public, matches the original spec); use **AlgoKit LocalNet** in Docker Compose for fast local iteration and CI.
- Backend wraps all chain calls in `services/blockchain/algorand\_client.py` — the rest of the app never touches `py-algorand-sdk` directly, so the chain can be swapped later without touching business logic.
- Customer-facing `/verify/[batchId]` page reads directly from chain + DB and renders the full custody chain, timestamps, and QR-scan history — this is your "trusted memory" story in the UI.

5. ML MODULE SPECS

Module 1 — Fraud & Anomaly Detection (primary research contribution)

- **Models:** Isolation Forest (unsupervised, scikit-learn) + XGBoost (supervised, trained on synthetically injected anomalies).
- **Experiment A features (baseline):** `supplier\_id, manufacturer\_id, medicine\_category, batch\_size, manufacturing\_location, distribution\_duration\_days, quantity\_ordered, quantity\_received, quantity\_discrepancy`
- **Experiment B features:** all of A **plus** `transfer\_count, ownership\_hops, route\_deviation\_score, timestamp\_gap\_hrs, verification\_fail\_count, unexpected\_entity\_flag` — all derived from on-chain transaction history.
- **H1:** Experiment B outperforms Experiment A on F1 and ROC-AUC. Test with McNemar's test, $\alpha = 0.05$. Log every run (params, metrics, confusion matrix) to MLflow so A vs. B is reproducible and citable.
- **Output:** real-time anomaly score 0-1 per transaction; score > 0.80 auto-raises an `audit\_alerts` row and pushes a WebSocket event to the admin dashboard.
- **Synthetic anomaly injection:** build `ml/synthetic/anomaly\_injection.py` — rule-based, 10-15% injection rate, patterns = unexpected ownership transfers, route deviations, abnormally short transfer intervals, quantity discrepancies. Tag every injected row

`is\_synthetic=True` and never let it leak into a "real fraud" claim anywhere in the UI copy.

Module 2 – Counterfeit Medicine Risk Scoring

- **Model:** Random Forest → Low (0-35%) / Medium (36-70%) / High (71-100%) + probability.
- **Features:** `supplier\_reliability\_score, batch\_size, manufacturing\_location\_risk\_index, chain\_verification\_status, transfer\_count, unexpected\_transfer\_flag, distribution\_duration\_deviation, complaint\_history\_count`.
- **Labels:** simulated from known counterfeit patterns (unregistered suppliers, abnormally short manufacturing time, route bypass, multiple unexpected transfers) – document this clearly as simulated ground truth, not real counterfeit confirmations.
- **Explainability:** SHAP values computed per prediction, stored in `ml\_counterfeit\_scores.shap\_values`, rendered as a bar chart on the batch detail page and admin dashboard.
- **Positioning:** this is the differentiator vs. all reviewed literature (Thomson & Varuna 2024, Parvathi et al. 2025) which require a physical sample/image – this module scores from metadata alone, before physical inspection is even possible.

Module 3 – Inventory Demand Forecasting

- **Models:** PyTorch LSTM vs. XGBoost Regression – pick the winner by MAE/RMSE on a held-out test split, log both to MLflow, deploy the better one (store which one and its metrics in `ml\_demand\_forecasts`).
- **Features:** `medicine\_id, region, month, historical\_demand (12-month rolling), seasonal\_flag, complaint\_rate, batch\_expiry\_proximity\_days, previous\_stockout\_flag`.
- **Output:** 30-day forecast per medicine per region, refreshed nightly via a Celery beat task, shown against current stock with a shortage-risk flag.

Dataset

Base: Kaggle "Blockchain-Based Medicine Supply Chain Management" (Bidyutmal Saha), extended with the synthetic anomaly/counterfeit generators above. Keep raw, synthetic, and feature-engineered versions in clearly separated files/tables.

6. API SURFACE (FastAPI, auto-documented at `/docs`)

...

POST	/auth/register	(admin creates participants; customers self-register)
POST	/auth/login	-> JWT access + refresh
GET	/participants/me	
POST	/raw-materials	(supplier)
POST	/medicines	(manufacturer)
POST	/batches	(manufacturer) -> mints on-chain
batch record + QR code		

```

POST    /batches/{id}/transfer          (custody change) -> writes DB +
Algorand tx, triggers ML rescoring
GET     /batches/{id}/history          (full provenance, chain + DB merged)
-> powers /verify page

POST    /orders                        (buyer role)
PATCH  /orders/{id}/status

GET     /verify/{batch_code}          (public, no auth) -> QR landing page
data

GET     /admin/alerts                 (live + historical audit_alerts)
GET     /admin/risk-heatmap
GET     /admin/forecasts
POST    /admin/alerts/{id}/resolve

GET     /ml/fraud/score/{transaction_id}
GET     /ml/counterfeit/score/{batch_id}
GET     /ml/demand/forecast?medicine_id=&region=
POST    /ml/experiment/run            (admin/research: kick off Experiment
A vs B run, returns MLflow run id)

WS      /ws/admin/alerts              (live push of new anomaly alerts)

```

— — —

Build this as the flagship page – sleek, dense-but-readable, dark-mode-first:

Use shadcn/ui + Tailwind, Recharts for charts, consistent design tokens (spacing, type scale, one accent color for "safe" states and one for

"risk" states) – treat this as the single screen a professor/recruiter will actually look at.

8. BUILD ORDER (phased, same sequencing logic as the original blueprint, adapted to the new stack)

****Phase 1 – Foundation (Weeks 1-3)****

1. Docker Compose skeleton: postgres, redis, api, web.
2. AlgoKit project scaffold, TestNet account, LocalNet in Compose for dev.
3. SQLAlchemy models + first Alembic migration.
4. FastAPI auth (JWT) + role-based dependency; Next.js login/register pages.

****Phase 2 – Core Supply Chain Flow (Weeks 4-7)****

5. Six role portals: CRUD for raw materials, medicines, batches, orders.
6. Custody-transfer endpoint writing DB + Algorand atomically (with reconciliation job).
7. QR generation on batch creation; public `/verify/[batchId]` page.
8. Transaction history viewer merging chain + DB data.

****Phase 3 – Data & ML Foundation (Weeks 8-10)****

9. Clean/load Kaggle dataset; build synthetic anomaly + counterfeit-label generators.
10. Feature engineering pipelines for all three modules; EDA notebooks.

****Phase 4 – ML Modules (Weeks 11-15)****

11. Module 1: train Isolation Forest + XGBoost, run Experiment A vs B, log to MLflow, run McNemar's test.
12. Module 2: Random Forest + SHAP integration.
13. Module 3: train + compare LSTM vs XGBoost Regression.
14. Wrap all three as in-process FastAPI routes under `/app/ml/`, loaded via a model registry at startup.

****Phase 5 – Integration & Dashboard (Weeks 16-18)****

15. WebSocket alert pipeline; Celery nightly forecast refresh.
16. Build the admin dashboard end-to-end (Section 7).
17. End-to-end tests across all six roles (Playwright).

****Phase 6 – Evaluation & Writeup (Weeks 19-20)****

18. Finalize A/B experiment results, generate plots/tables for the paper.
19. Deploy demo (Vercel + Railway + TestNet), record walkthrough video.
20. Write results section using MLflow-logged metrics as the source of truth.

9. RESEARCH FRAMING TO KEEP CONSISTENT THROUGHOUT

- ****Baseline:**** Abbas et al. (2020) – ML used only for drug recommendation; this system extends ML into active supply-chain security.

- ****Core research question:**** Do blockchain-derived behavioural features improve ML fraud-detection accuracy over conventional features alone?
- ****Novelty claims (only these, stated carefully):****
 1. First empirical A/B test of blockchain-behavioural features for pharma fraud detection.
 2. First metadata-only counterfeit risk scoring (no physical sample required).
 3. First system combining fraud detection + counterfeit scoring + demand forecasting in one blockchain-native ML layer.
- Always phrase outputs as ****risk/probability estimates****, never as proof or guarantees (see Section 0 constraints).

10. WHAT TO TELL THE CODING AI TO START (short version, if you want a shorter kickoff message)

> Build "PharmaChain Intelligence" – a pharma supply-chain platform with 6 roles (Admin, Supplier, Manufacturer, Wholesaler, Distributor, Customer) on Algorand TestNet (AlgoKit/Algorand Python smart contracts), a FastAPI + PostgreSQL backend handling business logic, blockchain writes, and in-process ML inference, and a Next.js 15 + TypeScript + Tailwind + shadcn/ui frontend. Three ML modules live under `app/ml/`: fraud/anomaly detection (Isolation Forest + XGBoost, with an Experiment A vs Experiment B feature-set comparison logged to MLflow), counterfeit risk scoring (Random Forest + SHAP, metadata-only), and demand forecasting (PyTorch LSTM vs XGBoost Regression). Start with Phase 1 from the build order: Docker Compose skeleton, AlgoKit scaffold, SQLAlchemy models, JWT auth, and the login/register flow.

This document assumes no institutional requirement to use ASP.NET/C#/MSSQL. If your course mandates that stack for grading, tell me and I'll produce the equivalent prompt with .NET/MSSQL swapped back in while keeping everything else (architecture, ML specs, research design, dashboard) identical.